解决方案. 把各处私有代码收拢成带锁的 `kernel/mbox.c`，按 Linux 的约定重写：请求长度字填 0；发送前清空 FIFO；只接受地址与本次请求相同的回复；以“整条消息成功”加“返回结构第一个字为 0”判断 GPIO 操作成败。电源控制也按 Linux 的做法：先 `GET_GPIO_CONFIG` 读出并保留极性，再设置，最后 `GET_GPIO_STATE` 回读。修复后的日志是 `ov5647: CAM_GPIO0 direction=output polarity=normal state=1`。如果某个固件确实不处理这个标签，兜底方案是自制 `dt-blob.bin`，让固件启动时就把 p133 置为输出高。

问题 2：BSC0 对 0x36 始终 NACK

日志输出.

```
ov5647: no reply at i2c0 0x36 (GPIO44/45), BSC raw=90000131 flags=131 \
        now=... div=2500 fsel=55 levels=3
（更早的内核把 raw 打成 -6ffffecf）
```

问题分析. 供电已经正常，BSC0 却在 repeated-START 和 STOP/START 两种读法下都报错。逐个字段看：`flags` 的 `0x100` 是 `ERR`（从机没应答），`0x200` 才是时钟拉伸超时；`div=2500` 是 250 MHz/100 kHz；`fsel=55` 表示 GPIO44/45 都是 ALT1；`levels=3` 表示 SDA、SCL 都是高电平，总线空闲、没有被谁拉住。控制器确实发出了事务，从机确实没应答——但仅凭这一行，还分不清是“传感器不在”还是“控制器这条路不通”。另外两个副产品：真 BCM2837 在 `TA`（transfer active）仍为 1 时撤掉 `I2CEN`，状态机可能冻结，QEMU 不模拟这一点；内核 `printf` 的 `%x` 把最高位为 1 的值当有符号数打印，`0x90000131` 成了 `-6ffffecf`。

解决方案. 出错时采用两段式中止：先保持 `I2CEN`、清 FIFO，等 NACK+STOP 让 `TA` 归零，再关控制器、清粘滞状态位；`%x` 改为无符号。为了回答“传感器在不在”，增加一个与 BSC 完全无关的 bit-bang 探测（问题 3）。

问题 3：bit-bang 探测时而 ACK、时而 NACK

日志输出.

```
camera: GPIO bitbang probe 0x36=ACK 0x10=NACK 0x1a=NACK      （某次启动，格式）
camera: GPIO bitbang probe 0x36=NACK 0x10=NACK 0x1a=NACK     （同一模块，另一次）
```

问题分析. bit-bang 探测把 GPIO44/45 临时切成开漏 GPIO，只发一个地址字节、读第九个时钟上的 ACK，不读写任何寄存器。它本该给出确定的答案，同一个模块的结果却不稳定，问题出在诊断代码自己：第九个时钟调用的是通用的 `bb_release()`，它释放 SDA 之后会等 SDA 回到高电平——这对数据位是对的，对 ACK 却正好相反，ACK 的定义就是从设备把 SDA 拉低。于是每次都白等 100 µs 超时，再去采样时从设备可能已经松开了 SDA。

解决方案. ACK 周期只把 SDA 切成输入，在 SCL 高电平的中间直接采样。修正以后结果稳定为 `0x36=ACK`。这个结论很有分量：bit-bang ACK 而 BSC NACK，说明传感器、地址、供电、上拉和排线都没问题，故障只在 BSC0 的复用或时序这一段。同时规定无论 bit-bang 结果如何，都继续执行问题 4 的恢复扫描。

问题 4：BSC 四档降速全部失败

日志输出.

```
ov5647: BSC retry 100000 Hz failed status=... now=... div=...   （格式）
ov5647: BSC retry 50000 Hz failed status=...
ov5647: BSC retry 25000 Hz failed status=...
ov5647: BSC retry 10000 Hz failed status=...
```